CEC 322 Lab 8: Bit Reversal in C and Assembly

Jianhua Liu

Spring 2026

Contents

8.1 Introduction . . . 1
8.2 A naïve bit-reversal algorithm . . . 1
8.3 An efficient bit-reversal algorithm . . . 3
8.3.1 The original algorithm . . . 3
8.3.2 Principle of the algorithm . . . 4
8.3.3 Modified C code for the 8-bit case . . . 5
8.3.4 Translation of the C code to assembly for the 8-bit case . . . 6
8.3.5 Modified C code for the 16-bit case . . . 7
8.4 Programming tasks (90 pts) . . . 7
8.4.1 PT 1. Debugging the `mp_bit_reverse_n_bit_naive_s` function (30 pts) . . . 8
8.4.2 PT 2. Finish the `mp_bit_reverse_16_bit_fast_s` function (60 pts) . . . 8
8.5 Submission of Lab Report (10 pts) . . . 8

8.1 Introduction

Bit reversal is an operation which reveres all the bits of an unsigned number. Take an 8-bit number $x = 0\text{b}b_7b_6b_5b_4b_3b_2b_1b_0$ as example, the reversed value will be $y = 0\text{b}b_0b_1b_2b_3b_4b_5b_6b_7$.

Bit reversal is used in many applications, and arguably the most famous one is the application in Fast Fourier Transform (FFT). Here, we don't go to the details of FFT, but we will look closely at the bit reversal algorithms and their implementations in both C and assembly.

In the CMSIC DSP library, bit reversal is implemented by table checking, which is fast but take a large chunk of DRAM. Here, we consider using algorithms to program functions to perform bit reversal.

To help you get started quickly, the base project is provided in the enclosed `fi4b_bit_reversal.zip` file. Just open the unzipped project in CubeIDE.

8.2 A naïve bit-reversal algorithm

Below, we provide a naïve bit-reversal algorithm for the least significant `n` bits of a 32-bit number, where `n` is from 2 to 32.

```c
uint32_t mp_bit_reverse_n_bit_naive_c(uint32_t x, int n) {
    // Make sure n is between 2 and 32.
    int j = n - 1;
    for (int i = 0; i < n/2; i++, j--) {
        uint32_t mask_i = 1U << i;
        uint32_t mask_j = 1U << j;
        uint32_t b_i = (x >> i) & 1U;
        uint32_t b_j = (x >> j) & 1U;
        x &= ~(mask_i | mask_j);
        x |= (b_i << j) | (b_j << i);
    }
    return x;
}
```

The advantage of the above code that it is based on a very simple (and naïve) algorithm that we can come up very easily.

The above code is fairly easy to translate to assembly code, shown as below:

```
@ uint32_t mp_bit_reverse_n_bit_naive_c(uint32_t x, int n) {
@     // Make sure n is between 2 and 32.
@     int j = n - 1;
@     for (int i = 0; i < n/2; i++, j--) {
@         uint32_t mask_i = 1U << i;
@         uint32_t mask_j = 1U << j;
@         uint32_t b_i = (x >> i) & 1U;
@         uint32_t b_j = (x >> j) & 1U;
@         x &= ~(mask_i | mask_j);
@         x |= (b_i << j) | (b_j << i);
@     }
@     return x;
@ }
@ uint32_t mp_bit_reverse_n_bit_naive_s(uint32_t x, int n)
@   Input:
@     * x -> r0, n -> r1
@   Output:
@     * r0
@   Local vars:
@     * r2 = 1u, r3 = j, r4 = i,
@     * r5 = mask_i, r6 = mask_j,
@     * r7 = b_i, r8 = b_j

mp_bit_reverse_n_bit_naive_s:
    push    {r4-r8, lr}
    ldr     r2, =1              @ 1u
    sub     r3, r1, #1          @ j = N-1
    ldr     r4, =0              @ i = 0
loop_n_s:
    cmp     r4, r1, ASR #1      @ i < N/2 ?
    bge     end_n_s             @ If No, end loop
        lsl     r5, r2, r4      @ mask_i
        lsl     r6, r2, r4      @ mask_j
        lsr     r7, r0, r3      @ x >> i
        and     r7, r2          @ b_i
        lsr     r8, r0, r3      @ x >> j
        and     r8, r2          @ b_j
```

```
        orr     r5, r6          @ t = mask_i | mask_j
        bic     r0, r5          @ x &= ~t
        lsl     r7, r3          @ b_i << j
        lsl     r8, r4          @ b_j << i
        orr     r7, r8          @ t = (b_i << j) | (b_j << i)
        orr     r0, r7          @ x |= t
    add     r4, #1              @ i++
    sub     r3, #1              @ j--
    b       loop_n_s
end_n_s:
    pop     {r4-r8, pc}
```

Note that:

- The assembly code is a simple translation from the C code.
- There are some bugs in the above assembly code in the indented part of the code during the translation, which lead to the failing to pass the unit test. You need to debug the code to pass the unit tests. This is **programming task 1**, which will be discussed below.
- Here are the tricks for debugging you can use:
  - Fully understand the C function. If needed, use some numbers to walk through the code.
  - Carefully compare the C code and the assembly code to see if the implementation of the assembly is consistent with the C code.

The above naïve code works as a baseline code. Yet, its computational efficiency is very low. In fact, the computational complexity of this algorithm is $O(N)$.

Below, let us introduce an efficient algorithm that can significantly improve the computational efficiency, which has a computational complexity of $O(\log_2 N)$

8.3 An efficient bit-reversal algorithm

8.3.1 The original algorithm

Two efficient 32-bit reversal algorithms which are similar to each other are provided in http://aggregate.org/MAGIC/#Bit%20Reversal in the form of C code. One of them is listed below with some minor modifications for data types.

```c
uint32_t mp_bit_reverse_32_bit_swar_c(uint32_t x) {
    uint32_t y = 0x55555555;
    x = (((x >> 1) & y) | ((x & y) << 1));
    y = 0x33333333;
    x = (((x >> 2) & y) | ((x & y) << 2));
    y = 0x0f0f0f0f;
    x = (((x >> 4) & y) | ((x & y) << 4));
    y = 0x00ff00ff;
    x = (((x >> 8) & y) | ((x & y) << 8));
    return ((x >> 16) | (x << 16));
}
```

Note that the above algorithm is provided in the SWAR (SIMD (single instruction, multiple data) Within A Register) form, and hence the *swar* in the name of the function.

8.3.2 Principle of the algorithm

The above code is elegant, but it may not be easily understandable. Now, we provide a detailed explanation of the above algorithm using variables with different number of bits. If you can understand the above code, there is no need to read this subsection. You can skip to the next one titled *Modified C code for the 8-bit case*.

8.3.2.1 The case of 2-bit numbers

Consider the simplest case, which has only two bits to swap. Here, we will perform a single step swap (reversal), the *group of 1* (one bit in a group), *distance 1* (see below) swap, on `x` = 0bb_1b_0, where two groups of *one* bit (group 0bb_0 and group 0bb_1) with group distance 1 (distance from b_0 to b_1 in 0bb_1b_0) are swapped in units of bi-group (a pair of group of 1). We start with `y` = 0x1 = 0b01, the least significant (LS) two bits of the value of `y` in Line 2 (0x55555555).

Step 1a. Use the following operations to swap: `(x >> 1)& y` = 0b0b_1 and `(x & y)<< 1` = 0b$b_0$0.

Step 1b. Use bitwise OR to combine the results of the above swap to obtain the 2-bit result: `x = ((x >> 1) & y)| ((x & y)<< 1)` = 0bb_0b_1.

Note that this is exactly the same operation of Line 3 of the code above. Note also that the above can be written as follows, with the addition of bitwise AND to guarantee the 2-bit final result:

`x = ((x >> 1)| (x << 1)))& 0x3`.

8.3.2.2 The case of 4-bit numbers

With 4 bits, we need to perform two swaps.

First, consider the above *group of 1, distance 1* swap again. Here `x` = 0b$b_3b_2b_1b_0$, and the swap is performed on *4* groups of one bit, paired to *two* bi-groups. Each bi-group starts on the even bit; so these bi-groups contain 0bb_3b_2 and 0bb_1b_0, respectively. We start with `y` = 0x5 = 0b0101, the LS 4 bits of the value of `y` given in Line 2.

Step 1a. Use the following operations to swap on two bi-groups: `(x >> 1)& y` = 0b0$b_3$0b_1 and `(x & y)<< 1` = 0b$b_2$0$b_0$0.

Step 1b. Use bitwise OR to combine the results of the above swap to obtain the 4-bit intermediate result: `x = ((x >> 1)& y)| ((x & y)<< 1)` = $b_2b_3b_0b_1$.

Again, this is the same operation of Line 3 of the code above.

Second, consider the *group of 2* (two bits in a group), *distance 2* swap on `x` = 0b$b_2b_3b_0b_1$. We have two groups of *two* elements, b_2b_3 and b_0b_1, and the distance between the corresponding elements in each is *2*. Here we start with `y` = 0x3 = 0b0011, the LS 4 bits of the value of `y` given in Line 4.

Step 2a. Use the following operations to swap on one bi-group: `(x >> 2)& y` = 0b00b_2b_3 and `(x & y)<< 2` = 0b$b_0b_1$00.

Step 2b. Use bitwise OR to combine the results of the above swap to obtain the 4-bit final result: `x = ((x >> 2)& y)| ((x & y)<< 2)` = 0b$b_0b_1b_2b_3$.

Note that this is exactly the same operation of Line 5 of the code above. Note also that the above can be written as follows, with the addition of bitwise AND to guarantee the 4-bit final result:

`x = ((x >> 2)| (x << 2)))& 0xF`.

8.3.2.3 The case of 8-bit numbers

From the above discussions, we can see that we need to perform three swaps for this case.

First, consider again the above *group of 1, distance 1* swap on `x` = 0b$b_7b_6b_5b_4$_$b_3b_2b_1b_0$. The only difference as compared to the 4-bit case is that we have 4 bi-groups here. We start with `y` = 0x55 = 0b0101_0101, the LS 8 bits of the value of `y` given in Line 2.

Step 1a. Use the following operations to swap on 4 bi-groups: `(x >> 1)& y` = 0b0$b_7$0b_5_0$b_3$0b_1 and `(x & y)<< 1` = 0b$b_6$0$b_4$0_$b_2$0$b_0$0.

Step 1b. Use bitwise OR to combine the results of the above swap to obtain an 8-bit intermediate result: `x = ((x >> 1)& y)| ((x & y)<< 1)` = 0b$b_6b_7b_4b_5$_$b_2b_3b_0b_1$.

Second, consider the following *group of 2, distance 2* swap on `x` = 0b$b_6b_7b_4b_5$_$b_2b_3b_0b_1$. We start with `y` = 0x33 = 0b0011_0011, the LS 8 bits of the value of `y` given in Line 4.

Step 2a. Use the following operations to swap on two bi-groups: `(x >> 2)& y` = 0b00b_6b_7_00b_2b_3 and `(x & y)<< 2` = 0b$b_4b_5$00_$b_0b_1$00.

Step 2b. Use bitwise OR to combine the results of the above swap to obtain another 8-bit intermediate result: `x = ((x >> 2)& y)| ((x & y)<< 2)` = 0b$b_4b_5b_6b_7$_$b_0b_1b_2b_3$.

Note that this is exactly the same operation of Line 4 of the code above.

Finally, consider the following *group of 4, distance 4* swap on `x` = 0b$b_4b_5b_6b_7$_$b_0b_1b_2b_3$. Here, we start `y` = 0x0F = 0b0000_1111, the LS 8 bits of the value of `y` given in Line 6.

Step 3a. Use the following operations to swap on one bi-group: `(x >> 4)& y` = 0b0000_$b_4b_5b_6b_7$ and `(x & y)<< 4` = 0b$b_0b_1b_2b_3$_0000.

Step 3b. Use bitwise OR to combine the results of the above swap to obtain the 8-bit final result: `x = ((x >> 4)& y)| ((x & y)<< 4)` = 0b$b_0b_1b_2b_3b_4b_5b_6b_7$.

Note that this is exactly the same operation of Line 7 of the code above. Note also that the above can be written as follows, similar to Line 10 with the addition of bitwise AND to guarantee the 8-bit final result:

`x = ((x >> 4)| (x << 4))& 0xFF`.

The above procedure of swap continues to the 16- and 32-bit cases.

8.3.3 Modified C code for the 8-bit case

Note that the above SWAR algorithm is especially useful if the processor supports SWAR. However, this form of C code is not easy to translate to the common ARM assembly instructions we have discussed in the class. Here, we provide a new C implementation for 8-bit reversals which is modified for quick and easy translation to ARM assembly instructions.

```c
uint8_t mp_bit_reverse_8_bit_fast_c(uint8_t x) {
    uint32_t y = 0x55;
    // x = (((x >> 1) & y) | ((x & y) << 1));
    uint32_t z = y & (x >> 1);
```

```
    x &= y;
    x = z | (x << 1);

    y = 0x33;
    // x = (((x >> 2) & y) | ((x & y) << 2));
    z = y & (x >> 2);
    x &= y;
    x = z | (x << 2);

    z = x >> 4;
    x = z | (x << 4);

    return x;
}
```

8.3.4 Translation of the C code to assembly for the 8-bit case

Below, we translate the above C code to assembly implementation. Note that a modified C version is also provided in the comments for a easy comparison between the implementations of the two languages.

```
@ uint8_t mp_bit_reverse_8_bit_fast_c(uint8_t x) {
@     uint32_t y = 0x55;
@     uint32_t z = y & (x >> 1);
@     x &= y;
@     x = z | (x << 1);
@     y = 0x33;
@     z = y & (x >> 2);
@     x &= y;
@     x = z | (x << 2);
@     z = x >> 4;
@     x = z | (x << 4);
@     return x;
@ }
@ uint8_t mp_bit_reverse_8_bit_fast_s(uint8_t x)
@   Input:
@     * x -> r0
@   Output:
@     * r0
@   Local vars:
@     * r1 = y, r2 = z
mp_bit_reverse_8_bit_fast_s:
    ldr     r1, =0x55               @ y = 0x55
    and     r2, r1, r0, lsr #1      @ z=y&(x>>1)
    and     r0, r1                  @ x &= y;
    orr     r0, r2, r0, lsl #1      @ x=z|(x<<1);
    ldr     r1, =0x33               @ y = 0x33
    and     r2, r1, r0, lsr #2      @ z=y&(x>>2)
    and     r0, r1                  @ x &= y;
    orr     r0, r2, r0, lsl #2      @ x=z|(x<<2);
    lsr     r2, r0, #4              @ z = x >> 4;
    orr     r0, r2, r0, lsl #4      @ x=z|(x<<4);
    ldr     r1, =0xFF               @ 8-bit mask
    and     r0, r1                  @ keep 8 bits
    bx      lr
```

Note that the last two lines of code before `bx lr` are used to make sure the return result is indeed 8 bits.

Without these two instructions, the Op2 `r0, lsl #4` in the previous instruction can make the number longer than 8 bits during left shifting as the register handles 32-bit numbers. Note that this is not strictly necessary as the return is DEFINED as 8 bit variable already.

Note that the above code can also perform bit reversal for the case of less than 8 bits. For example, to reverse a 5-bit number using the 8-bit algorithm, we just need to remove the least significant 8-5 = 3 bits after the reversal, as shown in the test function below:

```c
void test_reverse_5_bits_in_0x1D_with_8_bit_fast_c(void) {
    uint8_t A = 0x1D;    // A   = 0b0001_1101
    uint8_t exp = 0x17;  // exp = 0b0001_0111
mp_bit_reverse_8_bit_fast_c
    uint8_t act = mp_bit_reverse_8_bit_fast_c(A, 5) >> 3;
    TEST_ASSERT_EQUAL_UINT8(exp, act);
}
```

8.3.5 Modified C code for the 16-bit case

Below, we provide the C implementation for 16 bit reversal which is an extension to the above code of the 8-bit case.

```c
uint16_t mp_bit_reverse_16_bit_fast_c(uint16_t x) {
    uint32_t y = 0x5555;
    // x = (((x >> 1) & y) | ((x & y) << 1));
    uint32_t z = y & (x >> 1);
    x &= y;
    x = z | (x << 1);

    y = 0x3333;
    // x = (((x >> 2) & y) | ((x & y) << 2));
    z = y & (x >> 2);
    x &= y;
    x = z | (x << 2);

    y = 0x0f0f;
    // x = (((x >> 4) & y) | ((x & y) << 4));
    z = y & (x >> 4);
    x &= y;
    x = z | (x << 4);

    z = x >> 8;
    x = z | (x << 8);

    return x;
}
```

8.4 Programming tasks (90 pts)

The buildable base project is provided. Yet, some test functions cannot pass. This is intentional as passing these tests involves programming tasks (PT) for the project.

8.4.1 PT 1. Debugging the `mp_bit_reverse_n_bit_naive_s` function (30 pts)

You will see that the two tests for the `mp_bit_reverse_n_bit_naive_s` function cannot pass. However, if you compare the assembly code with the C code, you will see that there is no architectural error in the assembly code. (We have not learned the `for` loop in assembly yet. However, it is guaranteed that the structure of the program is ok.) The trouble is caused by some small bugs, usually including the following:

- A wrong number or instruction for shifting.
- A wrong instruction for an intended arithmetic or logic operation.
- A wrong register for an intended variable is.

Figure out the issue by checking the test results and reading the code carefully. Then, modify the assembly code to pass the unit tests for this function.

8.4.2 PT 2. Finish the `mp_bit_reverse_16_bit_fast_s` function (60 pts)

This function is an extension of the given `mp_bit_reverse_8_bit_fast_s` function. Read the given functions in C and assembly carefully and figure out the pattern. Then write the new function by extending the given `mp_bit_reverse_8_bit_fast_s` function to pass all the tests.

8.5 Submission of Lab Report (10 pts)

The lab report should at least include the following.

- The code snippets of all your own code. Note that you need to write the code with clear comments so that a fellow programmer can understand your code easily.
- A screenshot showing your results of pass all the unit tests.

The report and zip file names should be something like:

- `fi4b-report-lastname-firstname.pdf`.
- `fi4b-proj-lastname-firstname.zip`.